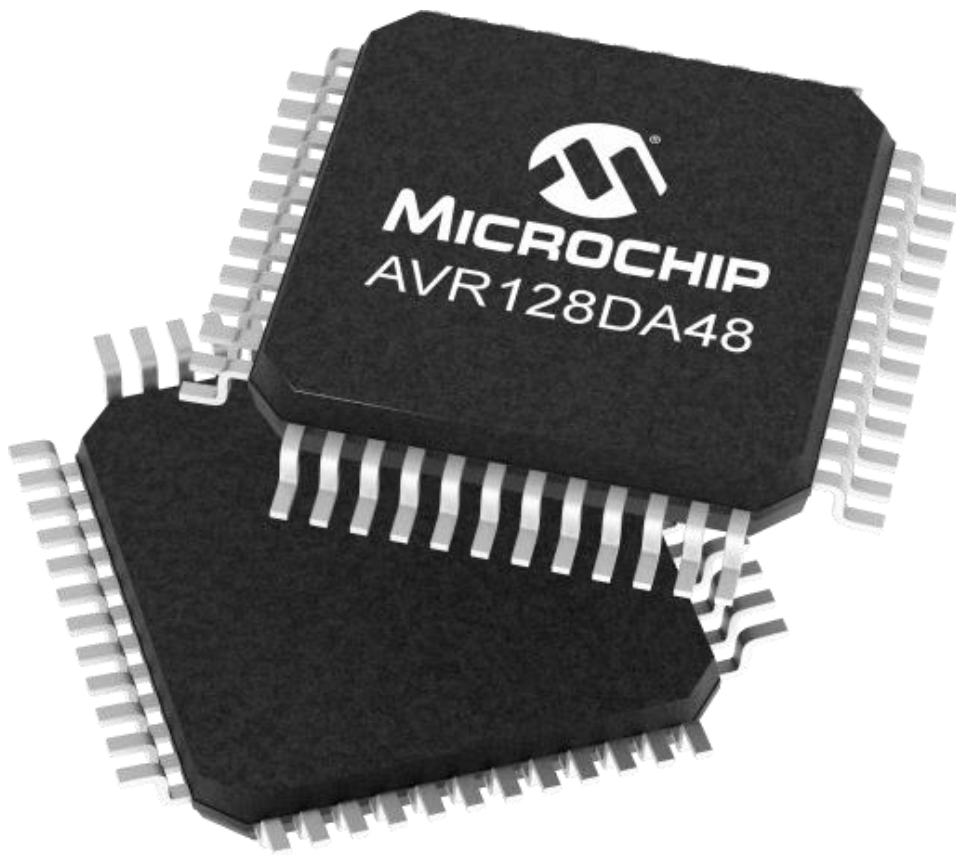


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 26 - Bare-Metal Driver Architecture



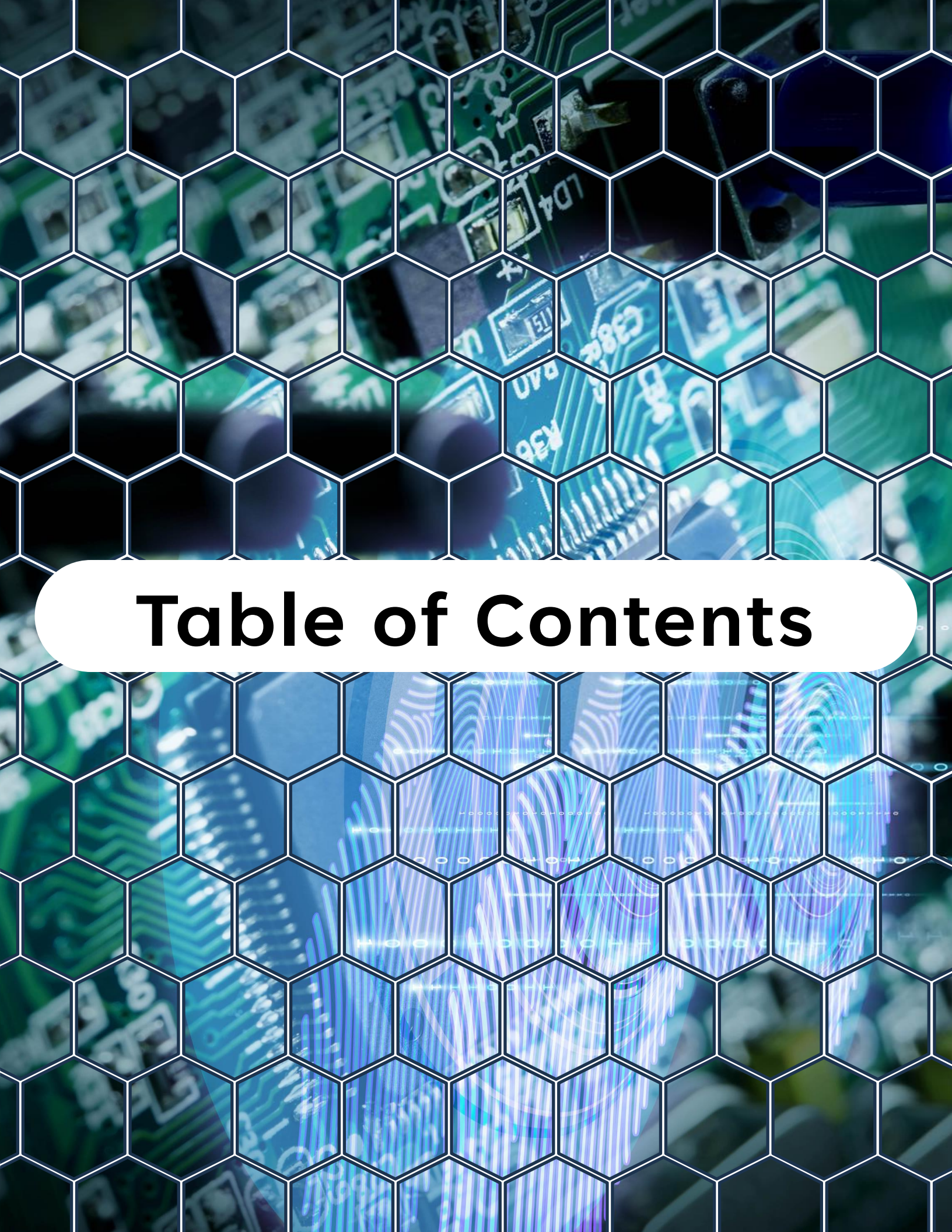
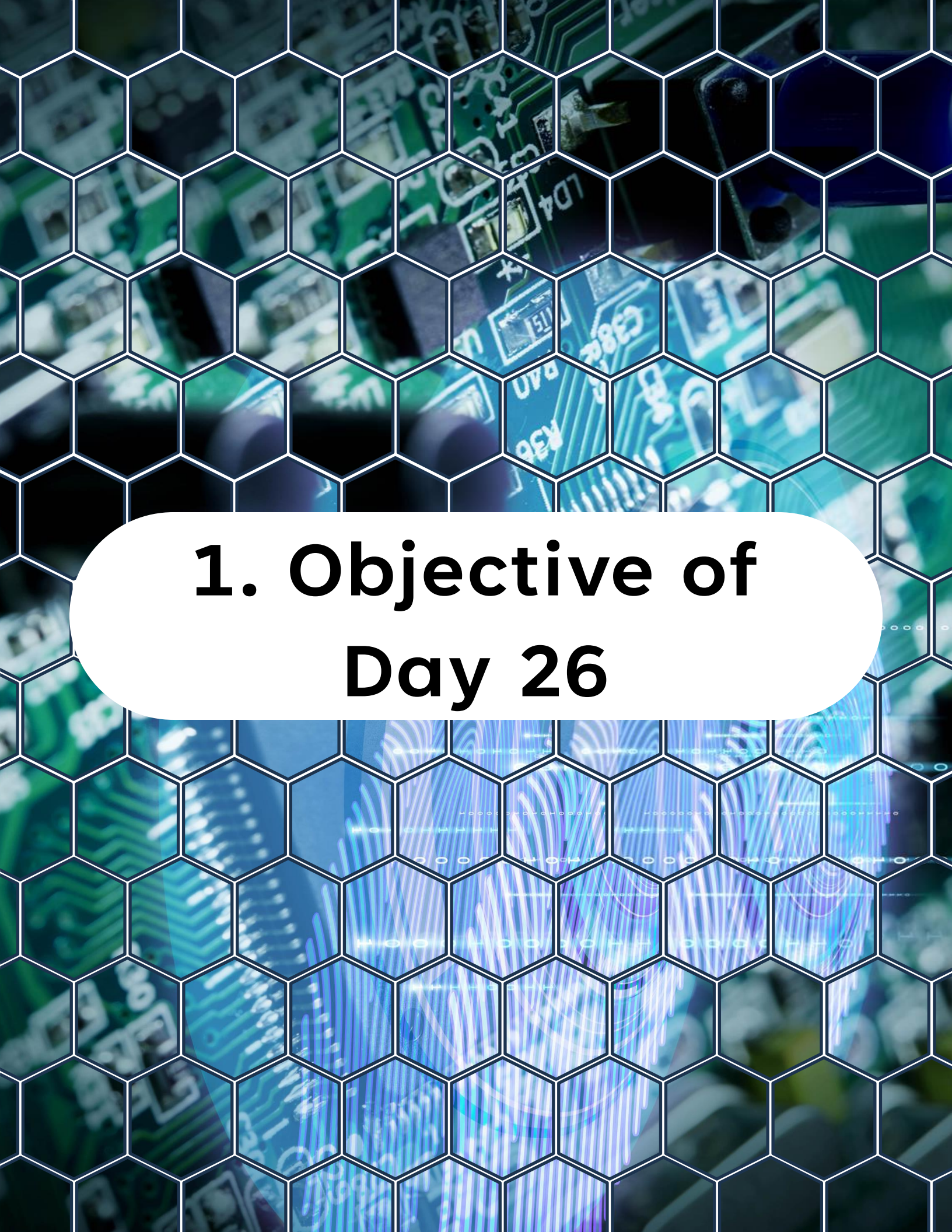


Table of Contents

Table of Contents

1. Objective of Day 26
2. What Is a Driver?
3. Why Driver Architecture Matters
4. Bare-Metal vs HAL
5. Layered Architecture
6. Driver Design Principles
7. Example - ADC Driver
8. Benefits of This Approach
9. Scaling the Design
10. Advanced Considerations
11. Common Mistakes
12. Lab - Refactor Existing Code into Driver
13. What You Should Understand Now
14. Preview of Day 27



1. Objective of Day 26

1. Objective of Day 26

Up to this point, you have learned how to configure peripherals directly using registers.

That is necessary.

But it is not sufficient for building real systems.

Today, we transition from writing code that works to writing code that scales, is reusable, and maintainable.

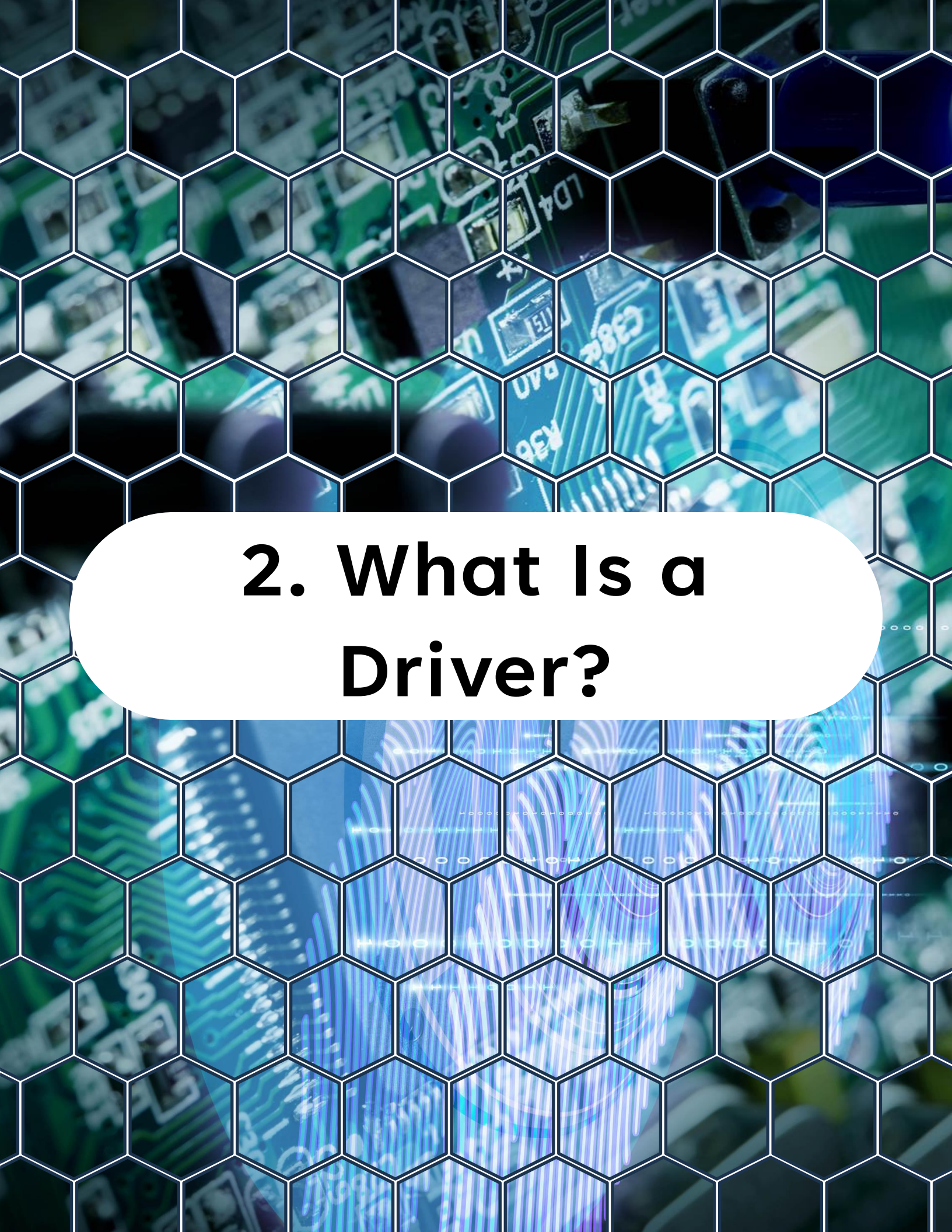
By the end of today, you will understand:

- What a driver is in embedded systems
- How to structure bare-metal drivers
- How to separate hardware access from application logic

1. Objective of Day 26

- How to design reusable and clean interfaces
- How to build a simple ADC driver module

This is where your firmware starts to look like a real project.



2. What Is a Driver?

2. What Is a Driver?

A driver is a software layer that:

- Configures hardware
- Provides an interface to the application
- Hides low-level register details

Instead of writing this everywhere:



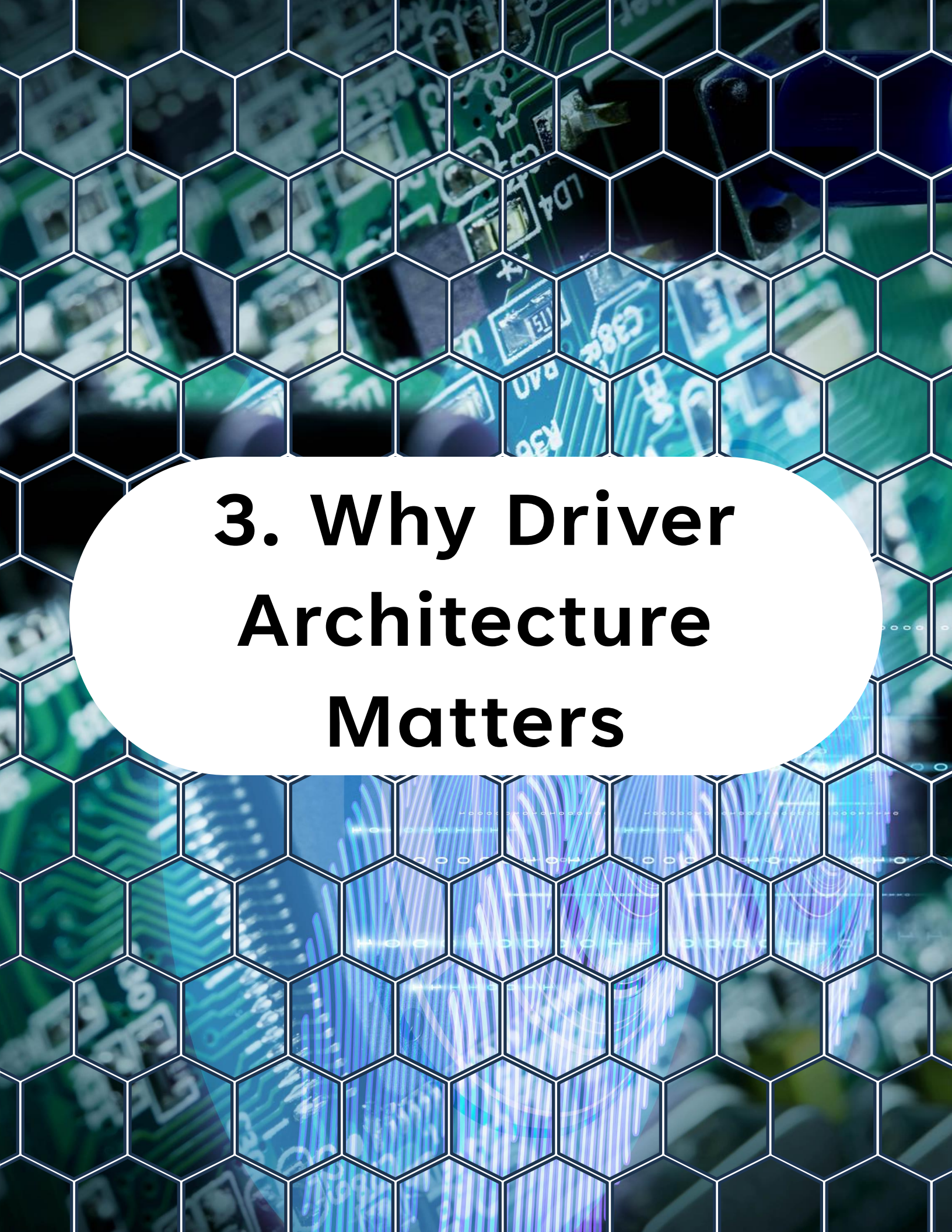
```
1  ADC0.CTRLA = ADC_ENABLE_bm;  
2  ADC0.COMMAND = ADC_STCONV_bm;
```

You write:



```
1  adc_start();
```

The driver encapsulates the hardware complexity.



3. Why Driver Architecture Matters

3. Why Driver Architecture Matters

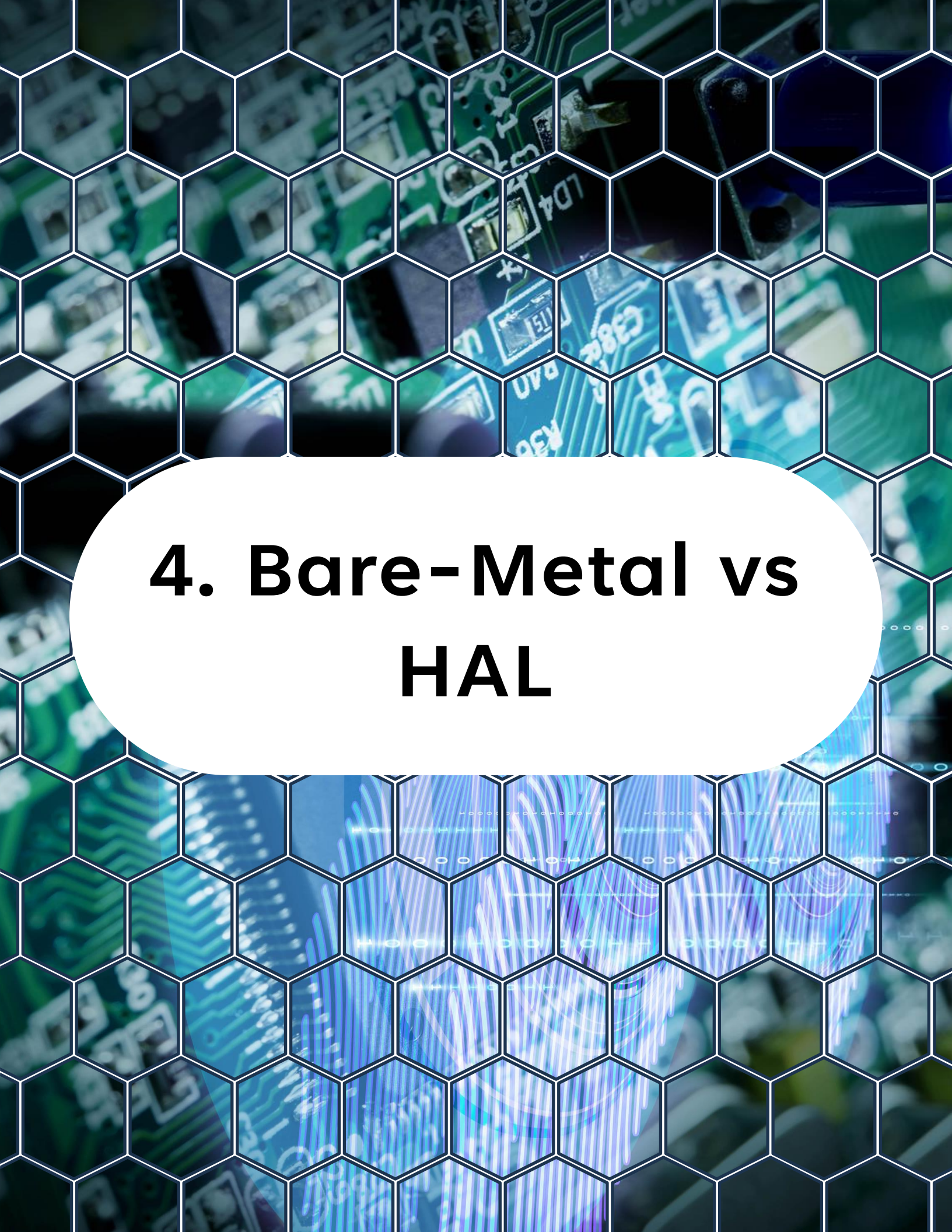
Without structure:

- Code is duplicated
- Registers are accessed everywhere
- Hard to maintain
- Hard to debug
- Not reusable

With drivers:

- Code is modular
- Easier to test
- Easier to reuse
- Easier to scale

This becomes critical as projects grow.



4. Bare-Metal vs HAL

4. Bare-Metal vs HAL

Important distinction:

HAL (Hardware Abstraction Layer)

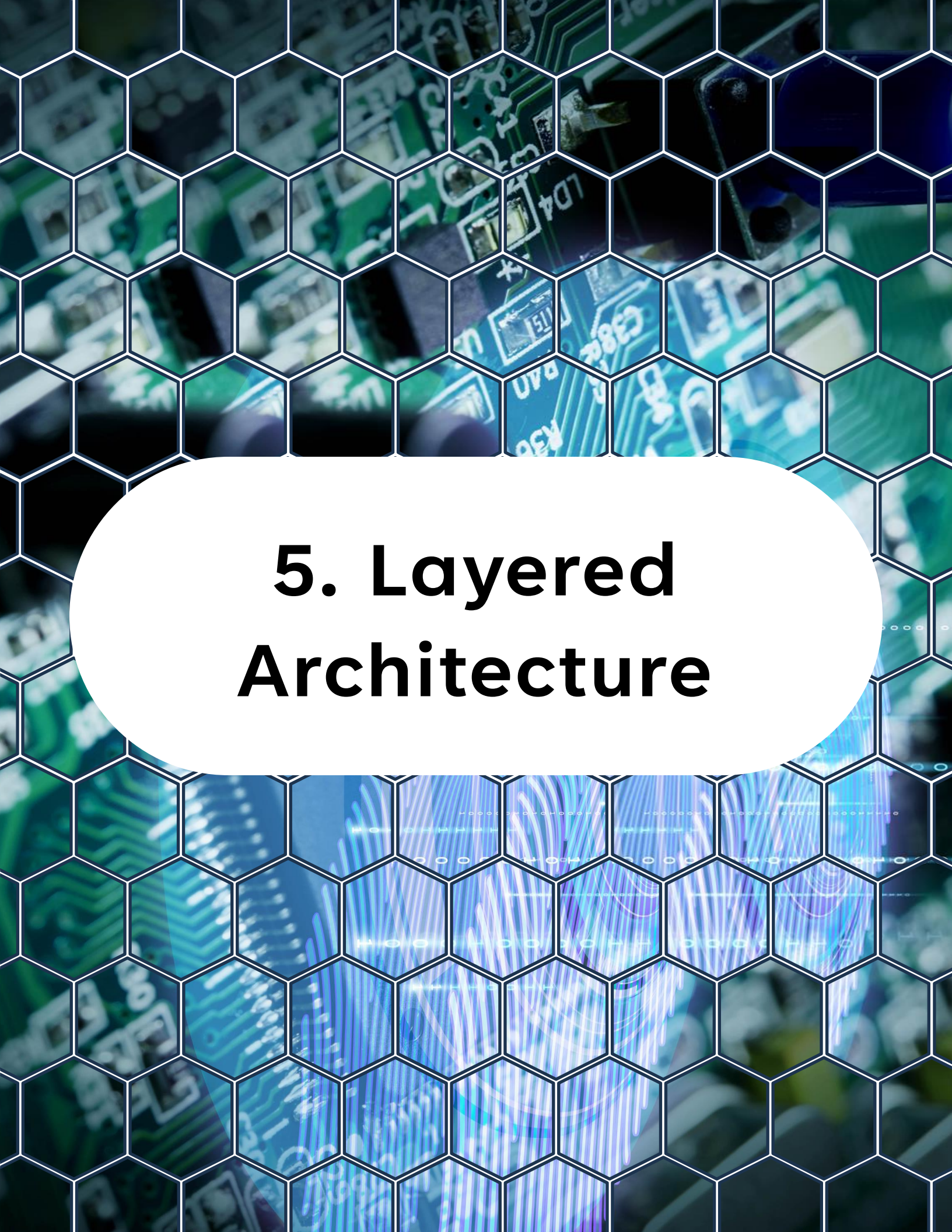
- Fully hides hardware
- Portable across MCUs
- Often heavy and slow

Bare-Metal Driver (our approach)

- Still uses registers directly
- Thin abstraction layer
- Minimal overhead
- Maximum control

We are building:

A lightweight, register-aware driver layer



5. Layered Architecture

5. Layered Architecture

A clean embedded architecture typically has:

Application Layer

Driver Layer

Hardware (Registers)

Responsibilities

Application:

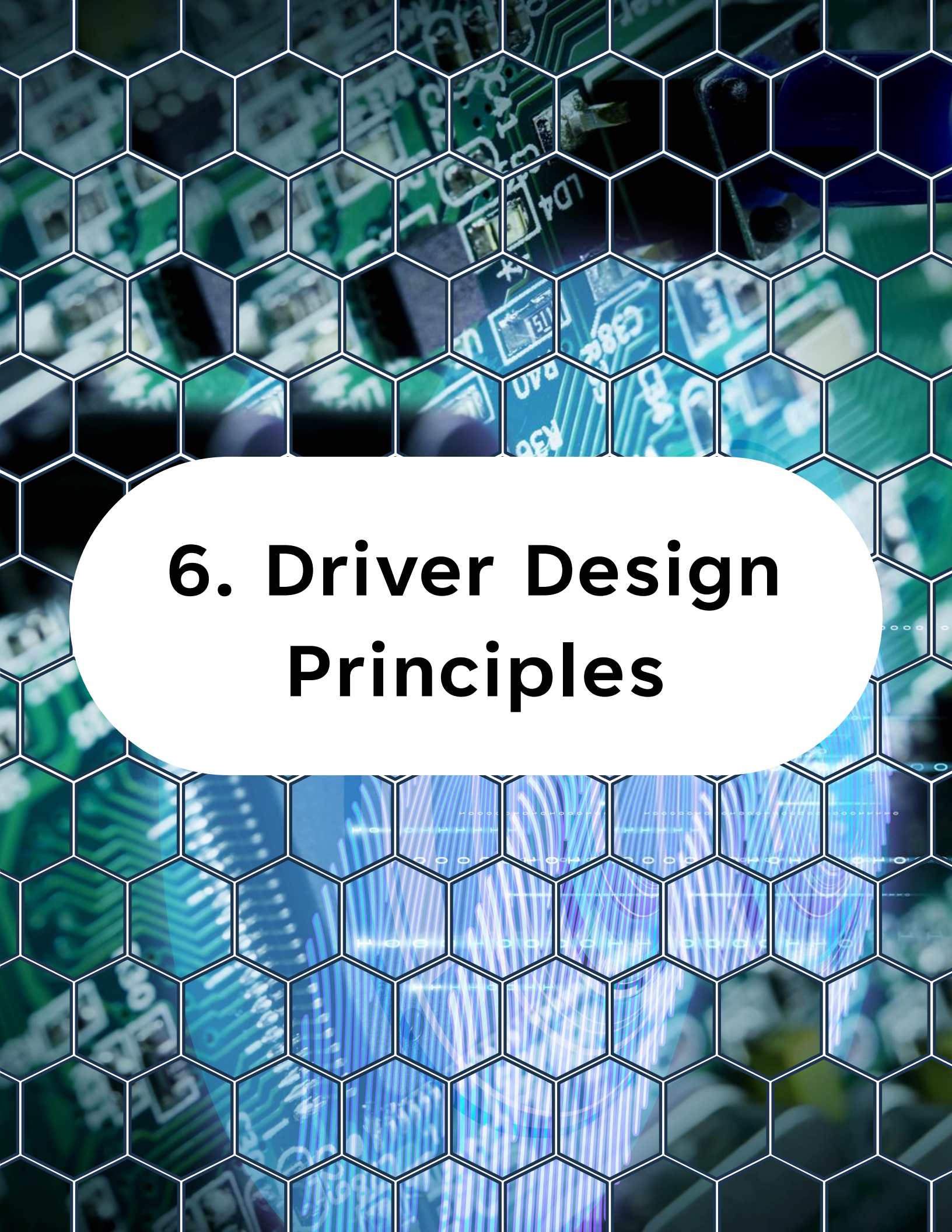
- Business logic
- Decision making

Driver:

- Peripheral control
- Hardware interface

Hardware:

- Registers and silicon



6. Driver Design Principles

6. Driver Design Principles

6.1 One Peripheral = One Module

Example:

- adc.c / adc.h
- uart.c / uart.h
- gpio.c / gpio.h

6.2 Clear Interface

The header file defines what the driver exposes:



```
1 void adc_init(void);  
2 uint16_t adc_read(void);
```

The application does not care about registers.

6. Driver Design Principles

6.3 Hide Register Access

All register-level code stays inside the driver.

Never do this in application:



```
1 ADC0.CTRLA = ...
```

6.4 Minimal State

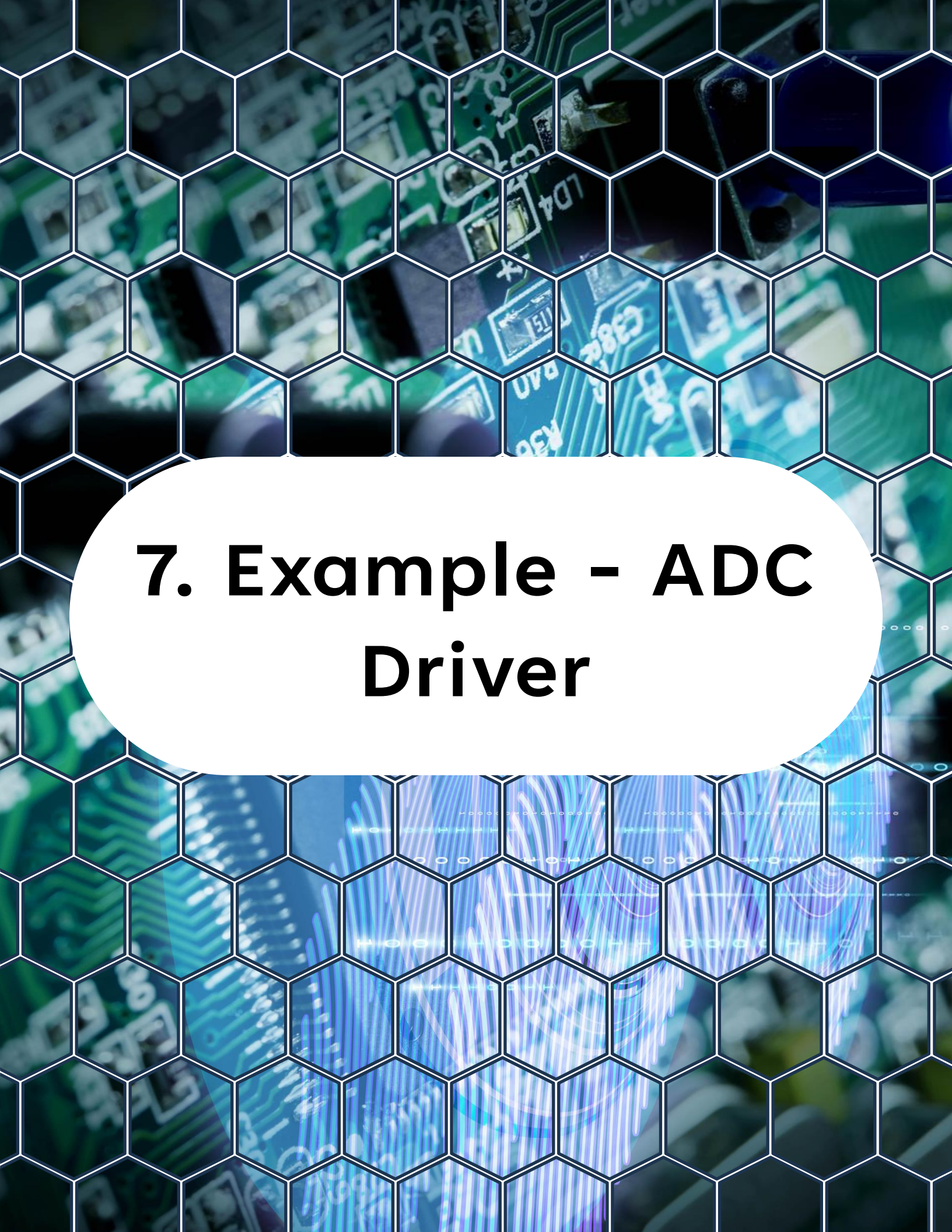
Keep drivers simple.

Avoid unnecessary global variables unless needed.

6.5 Deterministic Behavior

Drivers should behave predictably:

- No hidden delays, No unexpected blocking,
Clear timing behavior

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and surface-mount components. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center of the slide, there is a large white oval containing the title text.

7. Example - ADC Driver

7. Example - ADC Driver

We will now build a simple ADC driver using everything learned in previous days.

7.1 adc.h

```
1  /**
2   * @file adc.h
3   * @brief ADC driver interface.
4   */
5
6  #ifndef ADC_H
7  #define ADC_H
8
9  #include <stdint.h>
10
11 /**
12  * @brief Initialize ADC.
13  */
14 void adc_init(void);
15
16 /**
17  * @brief Read ADC value from AIN0.
18  *
19  * @return 12-bit ADC result.
20  */
21 uint16_t adc_read(void);
22
23 #endif /* ADC_H */
```

7. Example - ADC Driver

7.2 adc.c

```
1  /**
2   * @file adc.c
3   * @brief ADC driver implementation.
4   */
5  #include "adc.h"
6  #include <avr/io.h>
7
8  /**
9   * @brief Initialize ADC0 peripheral.
10  */
11 void adc_init(void)
12 {
13     // Configure VREF for ADC0: 1.024V reference,
14     // always on.
15     VREF.ADC0REF = 1 << VREF_ALWAYS_ON_bp
16                 | VREF_REFSEL_1V024_gc;
17
18     // CLK_PER divided by 16 for ADC clock.
19     ADC0.CTRLA = ADC_PRESC_DIV16_gc;
20
21     // Select ADC0 input channel: AIN0.
22     ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;
23
24     // Enable ADC0, 12-bit resolution, free-running
25     // mode disabled, run in standby disabled,
26     // right-adjusted result, single conversion mode.
27     ADC0.CTRLA = 1 << ADC_ENABLE_bp
28                 | 0 << ADC_FREERUN_bp
29                 | ADC_RESSEL_12BIT_gc
30                 | 0 << ADC_RUNSTBY_bp
31                 | 0 << ADC_LEFTADJ_bp
32                 | 0 << ADC_CONVMODE_bp;
```

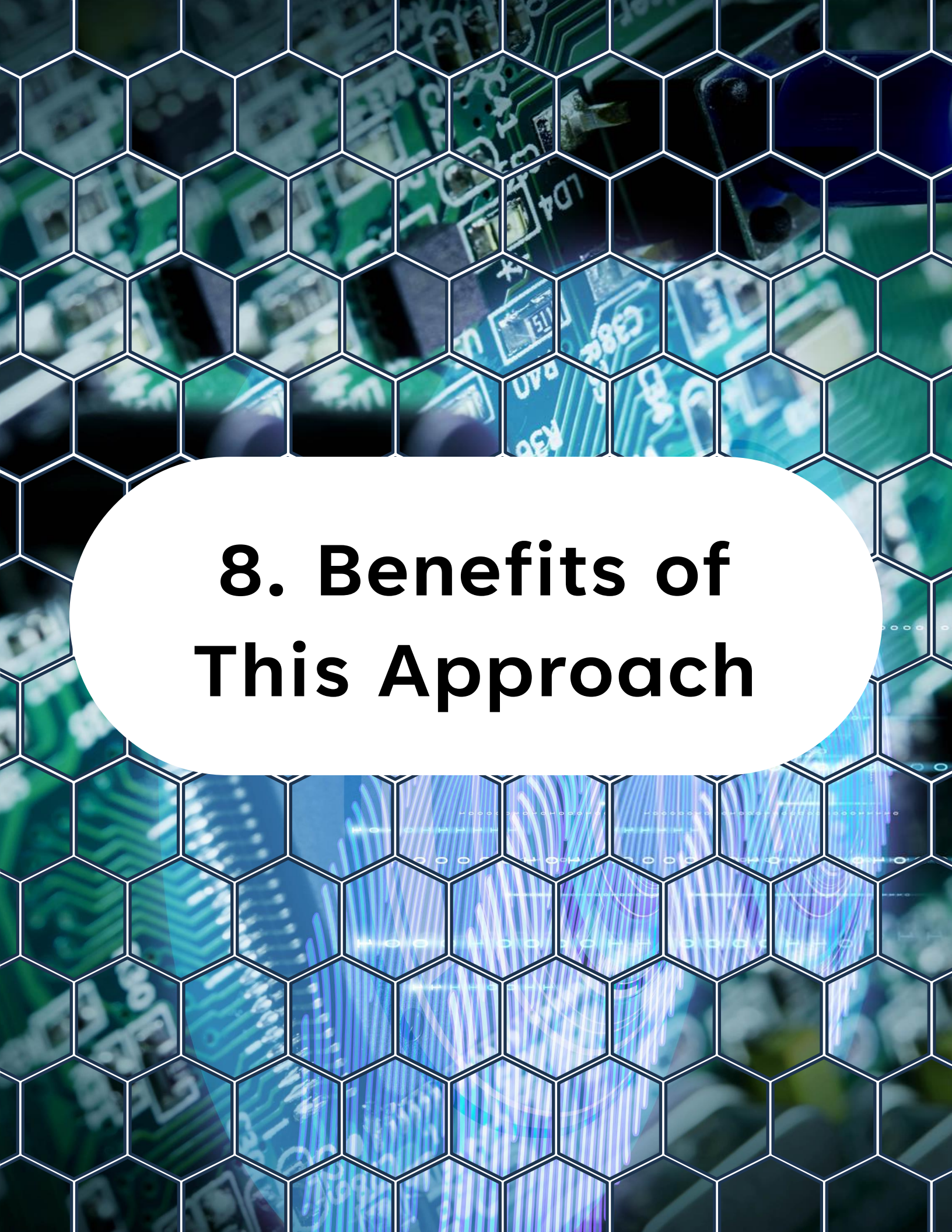

7. Example - ADC Driver

```
24 // Enable ADC0, 12-bit resolution, free-running
25 // mode disabled, run in standby disabled,
26 // right-adjusted result, single conversion mode.
27 ADC0.CTRLA = 1 << ADC_ENABLE_bp
28             | 0 << ADC_FREERUN_bp
29             | ADC_RESSEL_12BIT_gc
30             | 0 << ADC_RUNSTBY_bp
31             | 0 << ADC_LEFTADJ_bp
32             | 0 << ADC_CONVMODE_bp;
33 }
34
35 /**
36  * @brief Perform single ADC conversion.
37  */
38 uint16_t adc_read(void)
39 {
40     ADC0.COMMAND = ADC_STCONV_bm;
41
42     while (!(ADC0.INTFLAGS & ADC_RESRDY_bm))
43     {
44     }
45
46     ADC0.INTFLAGS = ADC_RESRDY_bm;
47
48     return ADC0.RES;
49 }
```

7. Example - ADC Driver

7.3 main.c (Using the Driver)

```
1  /**
2   * @file main.c
3   * @brief Application using ADC driver.
4   */
5
6  #include "adc.h"
7
8  int main(void)
9  {
10     uint16_t value;
11
12     // Initialize the ADC driver
13     adc_init();
14
15     while (1)
16     {
17         // Read a value from the ADC
18         value = adc_read();
19
20         /* Application logic here */
21     }
22 }
```



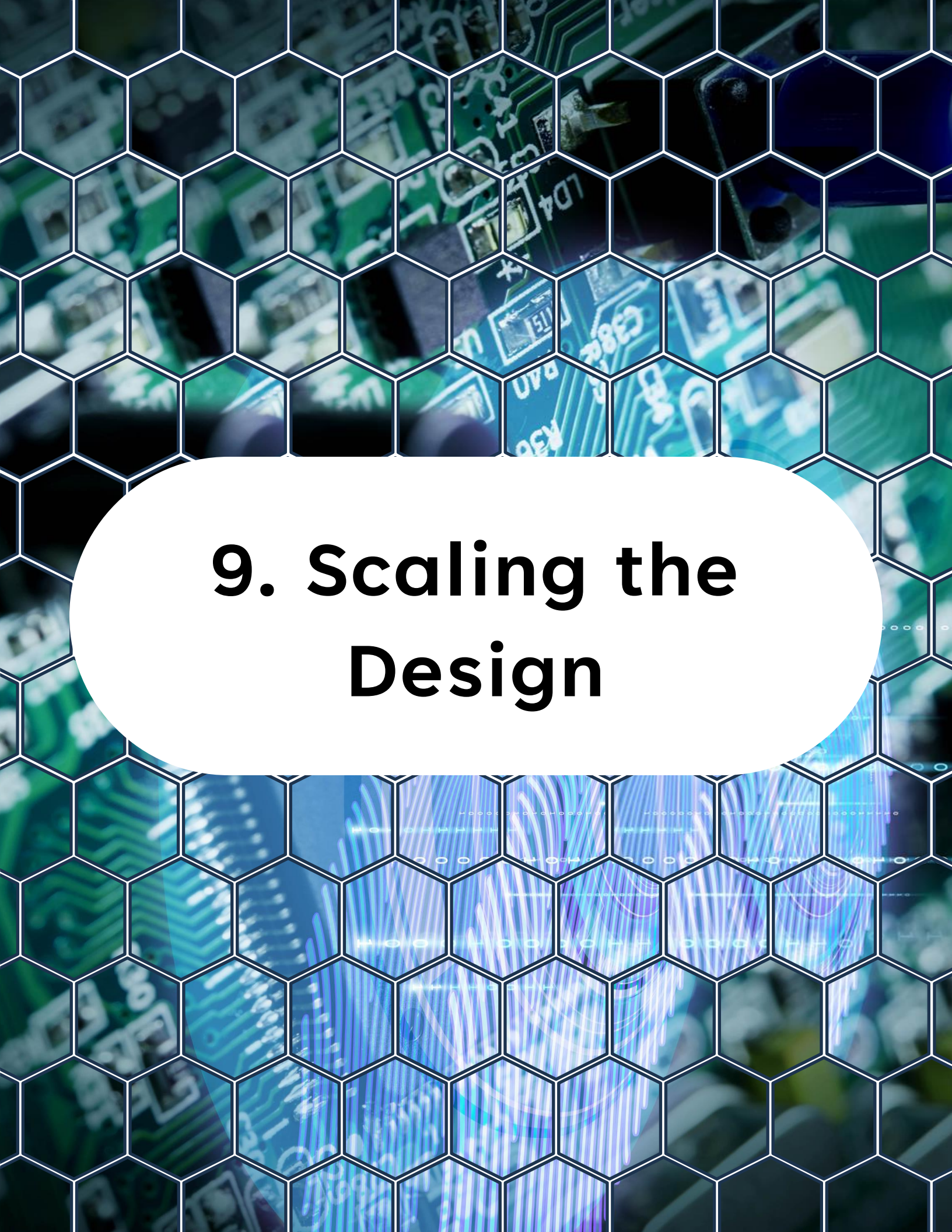
8. Benefits of This Approach

8. Benefits of This Approach

Now:

- ADC configuration is centralized
- Application code is clean
- Driver can be reused across projects
- Hardware changes affect only driver

This is how real firmware is structured.



9. Scaling the Design

9. Scaling the Design

As your project grows:

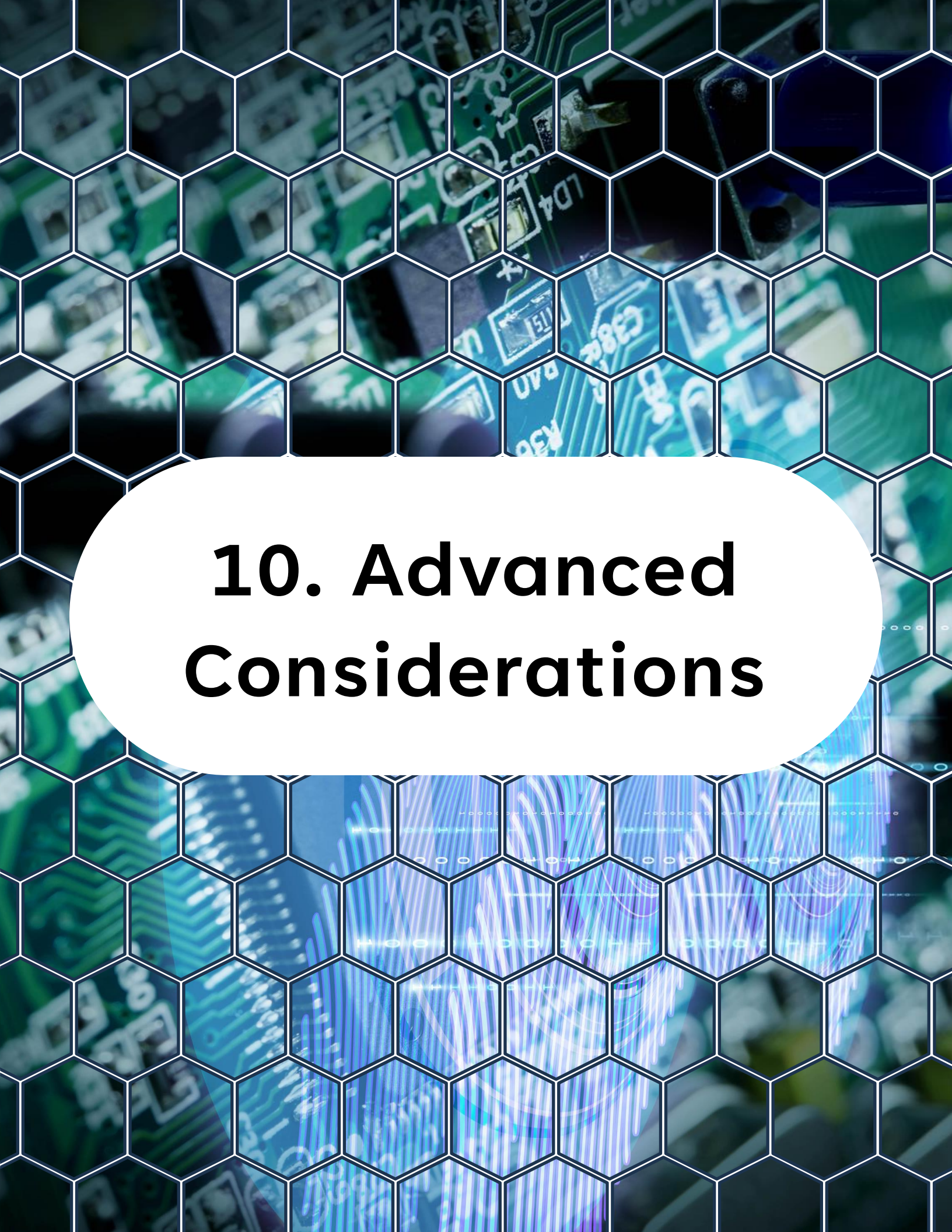
You will add:

- `uart.c`
- `spi.c`
- `timer.c`
- `eeeprom.c`

Each following the same pattern.

Eventually:

You build a complete reusable firmware framework.



10. Advanced Considerations

10. Advanced Considerations

10.1 Interrupt-Based Drivers

Instead of blocking:



```
1 uint16_t adc_read(void);
```

You can design:



```
1 void adc_start(void);  
2 bool adc_ready(void);  
3 uint16_t adc_get(void);
```

Or use callbacks.

10. Advanced Considerations

10.2 Configuration Structures

For flexibility:



```
1 typedef struct
2 {
3     uint8_t channel;
4     uint8_t prescaler;
5 } adc_config_t;
```

10.3 Static vs Global Scope

Use `static` where possible to limit scope and avoid conflicts.

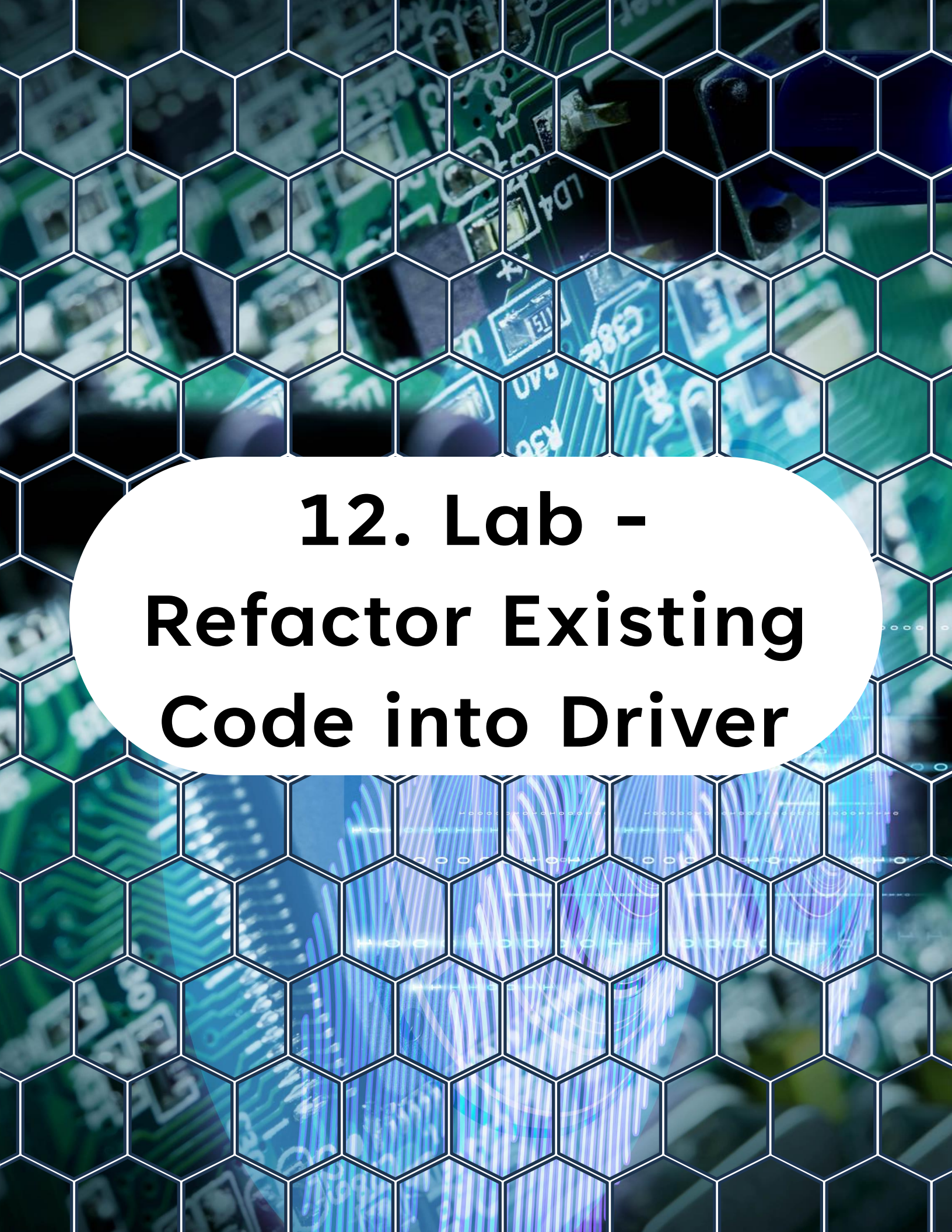


11. Common Mistakes

11. Common Mistakes

Avoid:

- Mixing application and driver code
- Exposing registers in headers
- Writing overly complex drivers
- Ignoring timing behavior
- Blocking unnecessarily

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C382', 'U7A', and 'K30'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white oval contains the main title text in a bold, black, sans-serif font.

12. Lab - Refactor Existing Code into Driver

12. Lab - Refactor Existing Code into Driver

Take your previous ADC example (Day 17 or 18) and:

1. Move ADC code into `adc.c` / `adc.h`
2. Remove all register access from `main.c`
3. Use driver API only

Optional:

Add LED driver as well.



13. What You Should Understand Now

13. What You Should Understand Now

You now know:

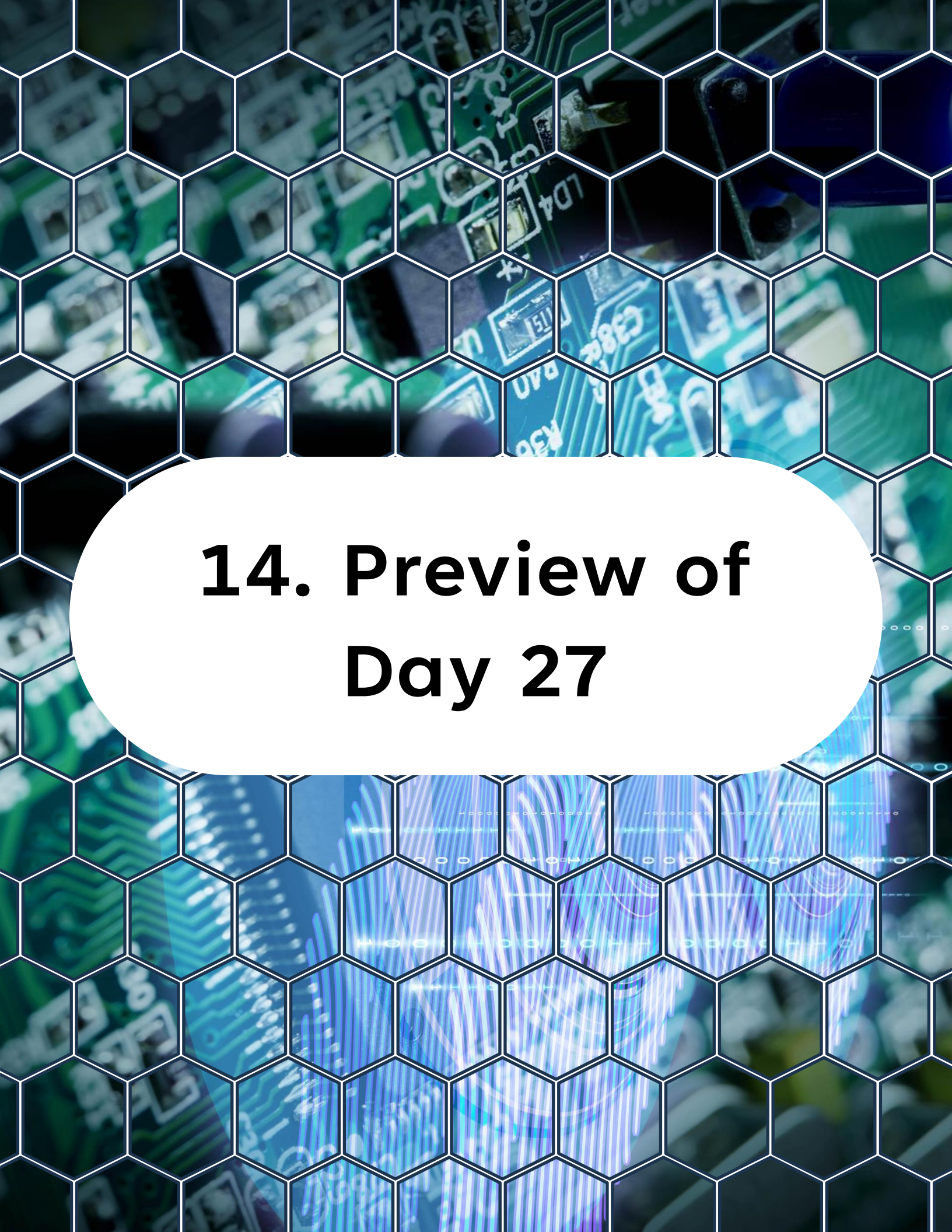
- What a driver is
- How to structure embedded code
- How to separate concerns
- How to build reusable modules
- How to design clean firmware

This is the transition from:

Writing code that works

to

Engineering firmware systems.



14. Preview of Day 27

14. Preview of Day 27

Next:

State Machines in Embedded Systems

You will learn how to:

- Structure application logic
- Handle complex workflows
- Build scalable control systems

This is where firmware becomes
architecture.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

<https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course>